Chapter 10. CNN Architectures, Residual Learning and Saliency

AlexNet, LRN, ZFNet, VGG, GoogLeNet/Inception, ResNet, saliency maps

Contents

1 Vocabulary and Costs **2**

2 AlexNet **2**

3 ZFNet and VGG **2**

4 GoogLeNet and the Inception Module **2**

4.1 Naive module 2

4.2 Bottleneck module 3

4.3 Split–transform–merge (2021 midterm Q5) 3

5 Network in Network: Exact versus Approximate Invariance (2022 midterm Q7) **3**

6 Why Depth Can Fail, and Residual Networks **4**

7 Saliency Maps **4**

1 Vocabulary and Costs

A network maps an image to class scores and then probabilities, $X \mapsto h(X) \mapsto \mathbf{s}(X) \in \mathbb{R}^Q \mapsto \mathbf{p}(X)$. For a convolution with D filters $K \times K \times C$ producing $H' \times W' \times D$:

$$N_{\text{param}} = D(K^2C + 1), \qquad N_{\text{act}} = H'W'D, \qquad N_{\text{MAC}} \approx H'W'DK^2C.$$

An FC layer $\mathbb{R}^n \to \mathbb{R}^m$ has $m(n + 1)$ parameters. Top-1 error asks whether the highest score is correct; top-5 error asks whether the true label is missing from the five highest scores.

2 AlexNet

Five convolutional and three FC layers (eight weight layers):

Stage	Settings	Output	
Input	RGB	$227 \times 227 \times 3$	
Conv1	96, 11×11, $S = 4$, $P = 0$	$55 \times 55 \times 96$	$((227 - 11)/4 + 1)$
Pool1, Norm1	3×3, $S = 2$; LRN	$27 \times 27 \times 96$	
Conv2	256, 5×5, $P = 2$	$27 \times 27 \times 256$	
Pool2, Norm2	3×3, $S = 2$; LRN	$13 \times 13 \times 256$	
Conv3, Conv4	384, 3×3, $P = 1$	$13 \times 13 \times 384$	
Conv5	256, 3×3, $P = 1$	$13 \times 13 \times 256$	
Pool3	3×3, $S = 2$	$6 \times 6 \times 256 = 9216$	
FC6, FC7, FC8	$9216 \to 4096 \to 4096 \to 1000$	1000 scores	

The FC layers dominate: $4096 \cdot 9217 = 37{,}752{,}832$, $4096 \cdot 4097 = 16{,}781{,}312$, $1000 \cdot 4097 = 4{,}097{,}000$, together 58,631,144 parameters, whereas Conv1 has $96(11^2 \cdot 3 + 1) = 34{,}944$. Design choices:

- **ReLU**: avoids sigmoid/tanh saturation on the positive side.
- **Large first stride**: $227 \to 55$ immediately reduces memory and computation (at the cost of detail).
- **Local response normalisation**: $a_{i,j,c} \mapsto a_{i,j,c} / \big(k + \alpha \sum_{r \in N(c)} a_{i,j,r}^2\big)^{\beta}$ normalises across neighbouring channels (not batch normalisation).
- **Dropout** 0.5 in the FC layers against overfitting.

3 ZFNet and VGG

ZFNet tunes AlexNet's kernel sizes and strides. VGG uses deeper stacks of small 3×3 convolutions: two stride-1 3×3 layers have a 5×5 receptive field with $18C^2$ weights (versus $25C^2$), three have 7×7 with $27C^2$ (versus $49C^2$), and interleaved ReLUs add nonlinearity. Receptive fields obey $r_\ell = r_{\ell-1} + (K_\ell - 1)j_{\ell-1}$, $j_\ell = S_\ell j_{\ell-1}$.

4 GoogLeNet and the Inception Module

4.1 Naive module

For input $28 \times 28 \times 256$, parallel branches with padding chosen to keep 28×28: 1×1 (128), 3×3 (192), 5×5 (96), 3×3 max pool (stride 1). Channel concatenation gives $128 + 192 + 96 + 256 = 672$ channels. Weights: $256 \cdot 128 + 9 \cdot 256 \cdot 192 + 25 \cdot 256 \cdot 96 = 1{,}089{,}536$.

4.2 Bottleneck module

Insert 1×1 reductions to 64 channels before the 3×3 and 5×5 convolutions and after pooling: outputs $128 + 192 + 96 + 64 = 480$ channels and

$$32{,}768 + (16{,}384 + 110{,}592) + (16{,}384 + 153{,}600) + 16{,}384 = 346{,}112 \text{ weights}$$

(about 330k), a 68% reduction.

Theorem 10.1: when a bottleneck saves weights

A direct $K \times K$ map $C \to D$ costs K^2CD; reducing to R channels first costs $CR + K^2RD$. This is cheaper iff $R < \dfrac{K^2CD}{C + K^2D}$ (for $K = 5, C = 256, D = 96$ the threshold is ≈ 231).

The 1×1 convolution is a channel-mixing matrix applied at every position (Chapter 9). GoogLeNet stacks such modules (22 weight layers) and replaces the large FC stack by global average pooling plus a final classifier.

4.3 Split–transform–merge (2021 midterm Q5)

A $256 \to 128 \to 128 \to 256$ conv–ReLU block (1×1, 3×3 with padding 1, 1×1, all with biases) has $32{,}896 + 147{,}584 + 33{,}024 = 213{,}504$ parameters. Thirty-two parallel $256 \to 4 \to 4 \to 256$ paths summed at the end have $32(1028 + 148 + 1280) = 78{,}592$ parameters; the 3×3 cost drops by the group factor 32. Implementation: three `nn.ModuleLists` of 32 convolutions, ReLU inside each path, then a sum (equivalently grouped convolutions with `groups=32`).

5 Network in Network: Exact versus Approximate Invariance (2022 midterm Q7)

NiN stacks mlpconv blocks (a spatial convolution followed by 1×1 convolutions, ReLUs, 3×3 stride-2 max pooling with ceil mode, dropout) and ends with average pooling over the final $8 \times 8 \times 10$ map. Take X with a single 1 at $(j, k) = (19, 19)$ and $\tilde{X}$ the same shifted to $(23, 19)$ (0-based, no biases, eval mode). Tracking where outputs can be nonzero: bias-free layers map zeros to zeros; a $K \times K$ convolution with padding $(K-1)/2$ widens the support by $(K-1)/2$; 1×1 convolutions and ReLU keep it; pool output p sees rows $2p, 2p+1, 2p+2$.

	rows for X	rows for $\tilde{X}$
conv1 (5×5)	17–21	21–25
out1 (16×16)	8–10	10–12
conv2 (5×5)	6–12	8–14
out2 (8×8)	2–6	3–7
out3 (3×3 conv)	1–7	2–8, truncated to 2–7

Columns stay 8–10, 2–6, 1–7. The shift 4 is a multiple of the total stride 4, so out1 is exactly shifted by 2 and out2 by 1; then out3 of $\tilde{X}$ equals out3 of X shifted down one row, except that row 7 of X's map would land on row 8, outside the 8×8 map. Hence the averaged outputs differ exactly by $\frac{1}{64}\sum_k \text{out3}(X)_{c,7,k}$: approximately equal, but not exactly equal in general.

6 Why Depth Can Fail, and Residual Networks

Deeper plain networks can have *higher training error*: this is not overfitting but an optimisation (degradation) problem. A deeper network that can represent identity layers contains the shallower one, so its *optimal* training loss is not worse, yet gradient methods may not find it. For $\mathbf{x}_{\ell+1} = f_\ell(\mathbf{x}_\ell)$, $\nabla_{\mathbf{x}_0} L = J_0^\top \cdots J_{L-1}^\top \nabla_{\mathbf{x}_L} L$; if every Jacobian has norm at most $\rho < 1$ the gradient shrinks like ρ^L ($0.9^{50} \approx 0.005$).

Theorem 10.2: residual block

$\mathbf{y} = \mathbf{x} + F(\mathbf{x}; \boldsymbol{\theta})$ gives $\dfrac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + J_F(\mathbf{x})$ and $\nabla_{\mathbf{x}} L = \mathbf{g} + J_F^\top \mathbf{g}$ with $\mathbf{g} = \nabla_{\mathbf{y}} L$.

The identity path carries the gradient even when J_F is small, and the block can represent the identity by $F = 0$: the network learns a residual $F = H - \mathrm{id}$ rather than the whole map H. This is a reparametrisation, not a guarantee (e.g. $J_F = -\mathbf{I}$ cancels the gradient; a ReLU after the sum alters the identity). If shapes differ, use a projection shortcut $\mathbf{y} = P(\mathbf{x}) + F(\mathbf{x})$ (e.g. a 1×1 or strided convolution). Unlike Inception's concatenation, residual addition keeps the channel count. A bottleneck block $256 \to 64 \to 64 \to 256$ (1×1, 3×3, 1×1) has $2 \cdot 256 \cdot 64 + 9 \cdot 64^2 = 69{,}632$ weights versus $18 \cdot 256^2 = 1{,}179{,}648$ for two full-width 3×3 layers. In the CIFAR-10 plots, residual networks remove the training-error degradation seen in plain networks. The full backpropagation formulas for a residual MLP are derived in Chapter 6 (2021 midterm Q7).

7 Saliency Maps

To see which pixels a prediction is sensitive to, differentiate the selected class score with respect to the input (not the weights):

$$G_{i,j,k} = \frac{\partial s_c(X)}{\partial X_{i,j,k}}, \qquad M_{i,j} = \max_k |G_{i,j,k}|.$$

By Taylor's theorem $s_c(X + \Delta X) \approx s_c(X) + \langle \nabla_X s_c(X), \Delta X \rangle$: a large $|G|$ means a small change of that pixel changes the score a lot, and the sign says in which direction. For $s(\mathbf{x}) = 2x_1 - 3x_2 + 0.1x_3$, the gradient $(2, -3, 0.1)$ ranks x_2 first. Saliency is local sensitivity, not a complete causal explanation.